Conciliación Bancaria Automatizada con Python y Pandas

Guía de estudio completa: conceptos, librerías, funciones,
flujo del notebook y del script, errores comunes y buenas prácticas

Proyecto: **concilicacion.ipynb** + **conciliacion.py** + **datos_prueba/**
Octubre de 2026 · Uso educativo

Índice

1. Objetivo del proyecto
2. Conceptos de conciliación bancaria
3. Librerías utilizadas y por qué
4. Funciones y métodos clave (glosario con ejemplos)
5. Flujo del notebook celda por celda
6. El script conciliacion.py
7. Datos de prueba y casos incluidos
8. Errores comunes y cómo evitarlos
9. Buenas prácticas
10. Resultados actuales del proceso
11. Siguientes pasos

1. Objetivo del proyecto

Automatizar la **conciliación bancaria** de una empresa: comparar el **libro auxiliar** (registros contables propios) con el **extracto bancario** (movimientos según el banco), y clasificar cada transacción para que un revisor humano pueda concentrarse solo en los casos problemáticos.

El proyecto tiene dos formas de ejecución con la misma lógica:

- **Notebook interactivo** (`concilicacion.ipynb`): ideal para explorar y aprender.
- **Script** (`conciliacion.py`): pide el nombre de un archivo de `datos_prueba/`, ejecuta el proceso completo y guarda el reporte.

El proceso cumple cinco funciones:

1. **Importar** ambas fuentes desde un Excel.
2. **Detectar duplicados** indicando en qué hoja ocurren (libro, banco o ambos).
3. **Detectar referencias sospechosas**: pares que casi coinciden pero difieren por espacios, mayúsculas o guiones.
4. **Clasificar** cada transacción: *Conciliado*, *Diferencia de Montos*, *Partida en Tránsito*, *Nota no Registrada* o *Revisión Manual*.
5. **Exportar** un reporte Excel con 4 hojas para revisión manual.

> **Principio del diseño:** el programa **detecta y reporta**, pero no corrige solo. La decisión final (borrar un duplicado, corregir una referencia) es humana y se aplica en el archivo de origen; luego se re-ejecuta el proceso.

2. Conceptos de conciliación bancaria

2.1 ¿Qué es la conciliación bancaria?

Es el proceso de comparar los movimientos registrados en la contabilidad de la empresa con los que reporta el banco, para asegurar que ambos coinciden. Las diferencias pueden ser por tiempos (una operación registrada en un lado y aún no en el otro) o por errores (montos distintos, registros duplicados, referencias mal escritas).

2.2 Las dos fuentes de datos

Fuente	Qué es	Columnas del proyecto
Libro_Auxiliar	Registros contables internos de la empresa (cheques girados, cobros, pagos).	`Fecha, Referencia, Concepto, Monto`
Extracto_Bancario	Movimientos según el banco (cobros de cheque, transferencias, comisiones).	`Fecha, Referencia, Descripcion_Banco, Monto`

2.3 Estados posibles de una transacción

Estado	Condición	Significado
Conciliado	Existe en ambos lados y los montos son iguales	La transacción cuadra; no requiere acción.
Diferencia de Montos	Existe en ambos lados pero los montos difieren	Hay que investigar: ¿error de digitación, comisión descontada, pago parcial?
Partida en Tránsito	Está en el libro y aún no en el banco	Ejemplo clásico: cheque girado que el beneficiario no ha cobrado.
Nota no Registrada	Está en el banco y aún no en el libro	Ejemplo: nota débito por comisión o cheque devuelto que la empresa no ha contabilizado.
Revisión Manual	Caso no previsto (red de seguridad)	Nunca debería activarse con datos limpios.

2.4 Duplicados y su origen

Una referencia repetida puede estar:

- **Solo en el libro**: se registró dos veces la misma operación.
- **Solo en el banco**: el banco procesó dos veces el mismo movimiento.
- **En ambos lados**: coinciden en estar duplicados.

Es importante detectar el duplicado **antes del merge** y en cada hoja por separado: si se busca sobre la tabla ya fusionada, un duplicado en una sola hoja "contamina" el resultado con filas repetidas (producto cartesiano) y ya no se puede saber dónde estaba el error.

2.5 Referencias sospechosas y normalización

El cruce se hace comparando texto exacto. `"CHQ-007"` y `"CHQ-007 "` son la misma operación, pero pandas las trata como distintas. La **normalización** convierte cada referencia a una forma canónica para compararlas de forma tolerante:

```
def normalizar(ref):
    return str(ref).strip().upper().replace(" ", "").replace("-", "")

"CHQ-007 "  →  "CHQ007"
"fac-1009"  →  "FAC1009"
"TRF103"    →  "TRF103"
```

Un **par sospechoso** es aquel cuya versión normalizada coincide pero cuyo texto original difiere: merece corrección en la fuente.

> La normalización actual NO ignora ceros a la izquierda: `CHQ-929` y `CHQ-0929` normalizan distinto (`CHQ929` vs `CHQ0929`) y por eso no se detectan como sospechosos. Es una mejora pendiente del proyecto.

2.6 Merge tipo outer y producto cartesiano

`pd.merge(..., how="outer")` es un *FULL JOIN*: conserva todas las filas de ambas hojas. Si una referencia no tiene pareja, la fila queda con los campos del otro lado en `NaN` (eso es lo que permite detectar partidas en tránsito y notas).

Ojo: si una referencia está *n* veces en el libro y *m* veces en el banco, el merge produce *n* × *m* filas. Por eso los duplicados deben revisarse antes de interpretar los totales.

3. Librerías utilizadas y por qué

Librería	Qué aporta al proyecto	Por qué se eligió
pandas	`DataFrame`, `read_excel`, `merge`, operaciones por columnas.	Es el estándar para datos tabulares en Python. Toda la lógica (comparar montos, marcar estados) se hace **vectorizada**, sin bucles, lo que es rápido y legible.
numpy	`np.where`, `np.select`, valor especial `NaN`.	Permite crear columnas con condiciones "fila a fila" de forma eficiente. `NaN` representa el "sin dato", clave para distinguir "falta el monto" de "monto = 0".
openpyxl	Motor de escritura de archivos `.xlsx`.	pandas lo usa internamente al llamar `to_excel` / `ExcelWriter` para generar el reporte final.

> pandas y numpy se importan con alias: `import pandas as pd` e `import numpy as np`, convención universal en la comunidad.

4. Funciones y métodos clave (glosario con ejemplos)

4.1 pandas

Función / método	Qué hace	Por qué se usa aquí
`pd.read_excel(ruta, hoja)`	Lee una hoja de Excel y la convierte en DataFrame.	Cada hoja del archivo de datos es una de las dos fuentes de la conciliación.
`pd.merge(a, b, on="Referencia", how="outer")`	Une dos tablas por una columna clave (JOIN).	Cruza libro y extracto por la referencia; `outer` conserva filas sin pareja.
`Series.duplicated(keep=False)`	Devuelve `True` en las filas cuya referencia ya apareció.	Detecta duplicados. `keep=False` marca **todas** las filas involucradas, no solo desde la segunda.
`Series.isna()` / `.notna()`	Indica si el valor es vacío (`NaN`) o no.	Distingue "falta el monto en este lado" de "el monto es 0".
`Series.eq(valor)`	Compara cada elemento con un valor (equivalente a `==`).	Comprobar si la celda contiene `"SI"` sin errores con `NaN`.
`Series.map(dict|función)`	Transforma cada valor según un diccionario o función.	Etiquetar `{True: "duplicado"}` y aplicar `normalizar()` a las referencias.
`Series.str.strip() / .upper() / .replace()`	Limpia texto: quita espacios extremos, pasa a mayúsculas, reemplaza caracteres.	Normalización de referencias para detectar pares sospechosos.
`DataFrame.assign(col=...)`	Devuelve una copia con una columna nueva.	Normalizar sin modificar los DataFrames originales (`df1`, `df2`).
`DataFrame.rename(columns={...})`	Renombra columnas.	Convierte los sufijos del merge (`Fecha_x`, `Monto_y`) en nombres legibles (`Fecha_Libro`, `Monto_Banco`).
`DataFrame.fillna("-")`	Reemplaza vacíos por un valor.	Deja el reporte final limpio (guion en vez de `NaN`).
`DataFrame.drop_duplicates()`	Elimina filas repetidas.	Quitar pares sospechosos repetidos en el reporte.
`DataFrame.loc[cond, columnas]`	Filtra filas y selecciona columnas.	Inspeccionar subconjuntos (p. ej. solo duplicados).
`Series.value_counts()`		Genera el *Resumen Ejecutivo* del reporte.

	Cuenta cuántas veces aparece cada valor.	
`Series.sum()`	Suma; sobre booleanos cuenta los `True`.	Contar "cuántos SI" hay en una columna.
`pd.concat([a, b])`	Apila DataFrames.	Unir `sin_libro` y `sin_banco` en una sola hoja del reporte.
`pd.ExcelWriter(...)` + `to_excel()`	Escribe múltiples hojas en un archivo Excel.	Generar el reporte con 4 hojas.

4.2 numpy

Función	Qué hace	Por qué se usa aquí
`np.where(cond, si, no)`	Elige entre dos valores fila por fila según una condición.	Crear columnas binarias: `"SI"/"NO"`, `np.nan` para las diferencias.
`np.select([c1, c2], [v1, v2], default)`	Evalúa varias condiciones en orden y asigna el primer valor que cumple.	Clasificación en cadena: `Estado_Final` y `Origen_Duplicado`. Sustituye a varios `if/elif` de forma vectorizada.
`np.nan`	Valor especial "sin dato".	Dejar vacías las celdas numéricas sin romper el tipo de la columna (a diferencia de `""`).

4.3 Módulos de la biblioteca estándar usados en el script

Módulo / función	Qué aporta	Por qué se usa
`pathlib.Path`	Rutas multiplataforma con el operador `/`.	Ubicar `datos_prueba/` de forma robusta.
`Path(__file__).resolve().parent`	Carpeta donde está el script.	Anclar rutas al script y no al directorio desde el que se ejecuta.
`Path.exists()` / `.glob("*.xlsx")`	Verificar existencia y listar archivos.	Validar el nombre ingresado por el usuario y mostrarle los disponibles.
`input()`	Leer texto del usuario.	Pedir el nombre del archivo a procesar.

5. Flujo del notebook celda por celda

Celda 1 — Importaciones

```
import pandas as pd
import numpy as np
```

Celda 2 — Lectura y marca de duplicados por hoja

```
df1 = pd.read_excel('Datos_Prueba.xlsx', 'Libro_Auxiliar')
df2 = pd.read_excel('Datos_Prueba.xlsx', 'Extracto_Bancario')

# Marcamos los duplicados ANTES del merge, en cada hoja por separado
# keep=False marca TODAS las filas involucradas en un duplicado
df1["Duplicado_Libro"] = np.where(df1["Referencia"].duplicated(keep=False), "SI", "")
df2["Duplicado_Banco"] = np.where(df2["Referencia"].duplicated(keep=False), "SI", "")
```

Los duplicados se marcan aquí, sobre cada hoja, para conservar la información de **dónde** está el problema.

Celda 3 — Referencias sospechosas

```
def normalizar(ref):
    return str(ref).strip().upper().replace(" ", "").replace("-", "")

tmp1 = df1[["Referencia"]].assign(Ref_Norm=df1["Referencia"].map(normalizar))
tmp2 = df2[["Referencia"]].assign(Ref_Norm=df2["Referencia"].map(normalizar))

sospechosas = tmp1.merge(tmp2, on="Ref_Norm", suffixes=("_libro", "_banco"))
sospechosas = sospechosas[
    sospechosas["Referencia_libro"] != sospechosas["Referencia_banco"]
].drop_duplicates()

sin_libro =
df1[~df1["Referencia"].map(normalizar).isin(df2["Referencia"].map(normalizar))]
sin_banco =
df2[~df2["Referencia"].map(normalizar).isin(df1["Referencia"].map(normalizar))]
```

Se usan copias temporales (`tmp1`, `tmp2`) para no ensuciar las fuentes.

Celda 4 — Merge

```
df = pd.merge(df1, df2, on="Referencia", how="outer")
```

El sufijo `_x` corresponde al libro y `_y` al banco (`Monto_x`, `Monto_y`, `Fecha_x`, `Fecha_y`).

Celda 5 — Coinciliado

```
df["Coinciliado"] = np.where(df["Monto_x"] == df["Monto_y"], "SI", "NO")
```

Celda 6 — Partida en Tránsito / Nota no Registrada

```
en_libro_sin_banco = df["Monto_x"].notna() & df["Monto_y"].isna()
en_banco_sin_libro = df["Monto_x"].isna() & df["Monto_y"].notna()

df["Partida en Tránsito / Nota no Registrada"] = np.select(
    [en_libro_sin_banco, en_banco_sin_libro],
    ["Partida en Tránsito", "Nota no Registrada"],
    default="")
```

Celda 7 — Diferencia de Montos

```
df["Diferencia de Montos"] = np.where(
    df["Monto_x"] - df["Monto_y"] != 0,
    df["Monto_x"] - df["Monto_y"], np.nan)
```

Detalle importante: el tercer argumento es `np.nan`, no `""`. Si se usara texto vacío, toda la columna pasaría a ser texto y los cálculos posteriores (`!= 0`, sumas) fallarían silenciosamente.

Celda 8 — Estado_Final y renombrado

```
condiciones = [
    (df["Monto_x"] == df["Monto_y"]),
    (df["Monto_x"].notna() & df["Monto_y"].notna() & (df["Monto_x"] !=
df["Monto_y"])),
    (df["Monto_x"].notna() & df["Monto_y"].isna()),
    (df["Monto_x"].isna() & df["Monto_y"].notna())]
opciones = ["Conciliado", "Diferencia de Montos", "Partida en Tránsito", "Nota no
Registrada"]
df["Estado_Final"] = np.select(condiciones, opciones, default="Revisión Manual")

df = df.rename(columns={
    "Fecha_x": "Fecha_Libro", "Concepto": "Concepto_Libro", "Monto_x": "Monto_Libro",
    "Fecha_y": "Fecha_Banco", "Descripcion_Banco": "Concepto_Banco", "Monto_y":
"Monto_Banco"})
```

`np.select` evalúa las condiciones **en orden**: gana la primera que se cumpla. Por eso la de "montos iguales" va antes que la de "montos diferentes".

Celda 9 — Origen del duplicado y reporte

```
df["Origen_Duplicado"] = np.select(
    [df["Duplicado_Libro"].eq("SI") & df["Duplicado_Banco"].eq("SI"),
     df["Duplicado_Libro"].eq("SI"),
     df["Duplicado_Banco"].eq("SI")],
    ["Duplicado en ambos lados", "Duplicado solo en Libro", "Duplicado solo en
Banco"],
    default="")

columnas_finales = ["Referencia", "Fecha_Libro", "Concepto_Libro", "Monto_Libro",
                    "Fecha_Banco", "Concepto_Banco", "Monto_Banco",
                    "Diferencia de Montos", "Estado_Final",
                    "Duplicado_Libro", "Duplicado_Banco", "Origen_Duplicado"]
df_reporte = df[columnas_finales].fillna("-")
```

Celda 10 — Exportación a Excel (4 hojas)

```
with pd.ExcelWriter(nombre_archivo_salida, engine='openpyxl') as writer:
    df_reporte.to_excel(writer, sheet_name="Detalle_Conciliacion", index=False)

    resumen = df_reporte['Estado_Final'].value_counts().reset_index()
    resumen.columns = ['Estado', 'Cantidad de Transacciones']
    resumen.to_excel(writer, sheet_name="Resumen_Ejecutivo", index=False)

    sospechosas.to_excel(writer, sheet_name="Referencias_Sospechosas", index=False)

    sin_libro_export = sin_libro[["Fecha", "Referencia",
"Monto"]].assign(Origen="Libro_Auxiliar")
    sin_banco_export = sin_banco[["Fecha", "Referencia",
"Monto"]].assign(Origen="Extracto_Bancario")
    sin_contrapartida = pd.concat([sin_libro_export, sin_banco_export],
ignore_index=True)
    sin_contrapartida.to_excel(writer, sheet_name="Sin_Contrapartida", index=False)
```

Celda 11 — Resumen en pantalla

```
print(f"{(df['Coinciliado'] == 'SI').sum()} Coinciliadas, "
      f"{(df['Partida en Tránsito / Nota no Registrada'] == 'Partida en
Tránsito').sum()} en tránsito, "
      f"{(df['Diferencia de Montos'].notna() & (df['Diferencia de Montos'] !=
0)).sum()} diferencias")
```

6. El script conciliacion.py

El script reproduce la lógica del notebook pero organizada en funciones, con entrada interactiva y rutas robustas. Se ejecuta con:

```
python conciliacion.py
```

Ejemplo de sesión:

```
Archivos disponibles en datos_prueba/:
  - Control_01.xlsx
  - Datos_Prueba_01.xlsx
Nombre del archivo a procesar (Enter = Datos_Prueba_01.xlsx):

Procesando: Datos_Prueba_01.xlsx
Pares sospechosos detectados: 6
¡Reporte generado con éxito! Archivo guardado como: datos_prueba/
Reporte_Datos_Prueba_01.xlsx

Resumen por estado:
Estado_Final
Conciliado              96
Partida en Tránsito     24
Nota no Registrada      24
Diferencia de Montos     8

96 Coinciliadas, 24 en tránsito, 8 diferencias
```

6.1 Estructura y funciones

Función	Responsabilidad	Detalles de diseño
`CARPETA_DATOS`	Constante con la carpeta de datos.	`Path(__file__).resolve().parent / "datos_prueba"`: anclada al script, funciona desde cualquier directorio.
`normalizar(ref)`	Forma canónica de una referencia.	`strip().upper().replace(" ", "").replace("-", "")`.
`pedir_archivo()`	Solicita y valida el nombre del archivo.	Lista los `.xlsx` disponibles, permite Enter para el archivo por defecto y falla con mensaje claro si no existe.
`cargar_datos(ruta)`	Lee ambas hojas y marca duplicados por hoja.	Envuelve la lectura en manejo de errores (`SystemExit` con el mensaje del problema).
`detectar_sospechosas(df1, df2)`	Pares sospechosos y sin contrapartida.	Usa copias temporales; devuelve 3 resultados.
`analizar(df1, df2)`	Merge y clasificación completa.	Devuelve `(df, df_reporte)`.
`exportar(ruta_entrada, ...)`	Genera el reporte de 4 hojas.	Nombre derivado del archivo de entrada: `Reporte_<nombre>.xlsx` dentro de `datos_prueba/`.
`resumen(df)`	Imprime indicadores.	Conteo por estado + línea resumen.
`main()`	Orquesta el flujo.	Protegido por `if __name__ == "__main__":`, lo que permite importar el módulo sin ejecutarlo (útil para tests).

6.2 Diferencias frente al notebook

- Las celdas que solo muestran `df` se reemplazan por `print()` (en un script no hay render automático de tablas).
- El reporte se guarda en `datos_prueba/` con nombre derivado del archivo de entrada, para no pisar reportes anteriores.
- El nombre del archivo de datos ya no está fijo: se pide por `input()`.

7. Datos de prueba y casos incluidos

El proyecto tiene dos conjuntos de datos:

7.1 Conjunto pequeño (raíz: `Datos_Prueba.xlsx`)

Hoja	Filas	Columnas
`Libro_Auxiliar`	25	`Fecha, Referencia, Concepto, Monto`
`Extracto_Bancario`	30	`Fecha, Referencia, Descripcion_Banco, Monto`

Casos incluidos: transacciones que cuadran, diferencias de montos (`FAC-1003`, `FAC-1006`, `FAC-1008`), partidas en tránsito (`CHQ-002`, `CHQ-003`), notas no registradas, duplicados en libro (`CHQ-005`, `CHQ-006`) y en banco (`TRF-102`, `NOM-1002`, `COM-202`), y errores de formato (`"CHQ-007 "`, `fac-1009`, `TRF103`, `CHQ-00A`).

7.2 Conjunto grande (`datos_prueba/Datos_Prueba_01.xlsx`)

120 filas por hoja, octubre de 2026, semilla 482916. Documentado en `datos_prueba/Control_01.xlsx`:

Caso incluido	Cantidad plantada	Qué valida
Duplicados solo en Libro_Auxiliar	4 referencias	`Duplicado_Libro`, origen "solo en Libro"
Duplicados solo en Extracto_Bancario	4 referencias	`Duplicado_Banco`
Duplicados en ambas hojas	2 referencias	Producto cartesiano 2×2 = 4 filas; origen "ambos lados"
Errores de formato de referencia	8 (6 detectables + 2 de ceros de más)	*Referencias sospechosas*
Pares con montos diferentes	8	Estado *Diferencia de Montos*
Partidas legítimas (solo libro)	15	*Partida en Tránsito*
Notas legítimas (solo banco)	15	*Nota no Registrada*
Referencias inválidas	2	Unilaterales para revisión manual

La tabla de control tiene una fila por cada error plantado, con la referencia, el monto y el resultado esperado; permite validar que el proceso detecta exactamente lo plantado.

7.3 Generar más conjuntos

`datos_prueba/PROMPT_GENERADOR_DATOS.md` contiene un prompt reutilizable con parámetros predeterminados para que un agente de código genere más conjuntos (`Datos_Prueba_02.xlsx`, `03`...) con su respectiva tabla de control, variando referencias y semilla.

8. Errores comunes y cómo evitarlos

Estos errores ocurrieron realmente durante el desarrollo de este proyecto:

#	Error	Mensaje / efecto	Corrección
1	Usar `if` con una Serie completa: `if df["Monto_x"] == df["Monto_y"]:`	`ValueError: The truth value of a Series is ambiguous…`	Usar `np.where` / `np.select` o combinar con `&`.
2	`np.where(cond, valor)` con solo 2 argumentos	`ValueError: either both or neither of x and y should be given`	Siempre 3 argumentos: `np.where(cond, si, no)`.
3	Mezclar números con `""` en una columna	La columna queda como texto; conteos como `!= 0` dan resultados falsos	Usar `np.nan` para vacíos numéricos.
4	`.sum` sin paréntesis	Imprime `<bound method…>` en vez del número	Llamar siempre al método: `.sum()`.
5	Asumir que `NaN != 0` es `False`	Las filas sin monto se cuentan como "diferencia"	Exigir valor en ambos lados: `notna() & (diff != 0)`.
6	Usar `and` / `&&` / `AND` entre condiciones de Series	`SyntaxError` o `ValueError`	En pandas se usa `&` (y), `|` (o), `~` (no), con cada condición entre paréntesis.
7	Comparar con `None`: `== None`, `NONE`	`NameError` o comparación sin sentido	Usar `.isna()` / `.notna()`.
8	Dos asignaciones seguidas a la misma columna para dos etiquetas	La segunda pisa a la primera (solo queda una categoría)	Una sola asignación con `np.select`.
9	Encadenar asignaciones con `|`: `A = B | C = D`	`SyntaxError` / `TypeError`	Combinar condiciones con `|`, no asignaciones.
10	Comillas dobles anidadas en f-strings con Python < 3.12	`SyntaxError`	Usar comillas simples dentro: `f"{(df['Col'] == 'SI').sum()}"`, o actualizar Python.
11	Comparar referencias sin normalizar	Pares iguales aparecen como partidas separadas	Normalizar (`strip` / `upper`) antes de comparar.
12	`duplicated()` sin `keep=False`	La primera fila del duplicado no se marca	`duplicated(keep=False)` marca todas las filas involucradas.
13	Merge sin `how="outer"`	Se pierden las filas que no coinciden (inner join por defecto)	Usar `how="outer"` para conservar ambos lados.
14	Corregir el DataFrame fusionado en vez del Excel de origen	El error reaparece en la siguiente ejecución	Corregir en la fuente y re-ejecutar el proceso.

15	Rutas relativas como `"datos_prueba"` en un script	Falla si se ejecuta desde otro directorio	Anclar con `Path(__file__).resolve().parent`.
16	Sobrescribir archivos de datos generados al preparar pruebas	Pérdida del conjunto de prueba	Respaldo previo y verificación antes de copiar.

9. Buenas prácticas

- **Vectorizar, no iterar:** nunca recorrer filas con `for` cuando existe una operación por columnas (`np.where`, `&`, `map`).
- **Detectar antes de unir:** los duplicados se marcan en cada hoja antes del merge; así no se pierde el origen del problema.
- **Normalizar para comparar, sin destruir el dato original:** se normaliza una copia; el dato crudo se conserva para el reporte.
- **Separar detección de corrección:** el programa sugiere; la decisión es humana y se aplica en la fuente.
- **Vacíos numéricos con `NaN`, nunca con texto** (`""`, `"-"`): el guion solo se usa al final, en el reporte, con `fillna("-")`.
- **Paréntesis en condiciones compuestas:** `(a) & (b)`, porque `&` tiene mayor precedencia que `==`.
- **Ordenar las condiciones de `np.select`:** primero las más específicas (montos iguales antes que "diferentes").
- **Organizar el script en funciones** con `main()` protegido por `if __name__ == "__main__":` para poder importarlo y probarlo.
- **Re-ejecutar desde cero tras cambiar datos:** Kernel → Restart & Run All, para no arrastrar estado viejo en memoria.
- **Respaldar los datos de prueba** antes de modificarlos o reemplazarlos.
- **Validar contra una tabla de control:** cada error plantado en los datos debe poder verificarse automáticamente contra el resultado esperado.

10. Resultados actuales del proceso

Con el conjunto pequeño (38 filas tras el merge):

```
18 Coinciliadas, 6 en tránsito, 3 diferencias
```

Con el conjunto grande `Datos_Prueba_01.xlsx` (152 filas tras el merge):

Indicador	Valor
Filas tras el merge	152
Conciliadas	96
Partidas en Tránsito	24
Notas no Registradas	24
Diferencias de Montos	8
Duplicados (filas marcadas)	8 en libro + 8 en banco + 8 en ambos
Pares sospechosos	6 (de 8 planteados; los 2 de ceros de más no son detectables aún)
Sin contrapartida	18 en libro + 18 en banco

```
96 Coinciliadas, 24 en tránsito, 8 diferencias
```

11. Siguientes pasos

- Aplicar las correcciones decididas en los archivos de origen y verificar que los estados vuelvan a cuadrar.
- Comparar montos con tolerancia de decimales (`round(2)` o `np.isclose`) para montos con centavos flotantes.
- Normalizar ignorando ceros a la izquierda para detectar casos como `CHQ-929` vs `CHQ-0929`.
- Detectar duplicados "parciales": misma referencia pero distinto monto o fecha (hoy solo se comparan montos).
- Agregar gráficos del resumen (barras por estado) con matplotlib.
- Aceptar el archivo de datos como argumento de línea de comandos (`python conciliacion.py archivo.xlsx`), además del `input()`.
- Convertir las validaciones en tests automáticos que lean la tabla de control y comparen contra lo esperado.